

BLACK HAT USA 2026

THREATFOREST: AUTOMATED ATTACK TREES FROM SOURCE CODE

Cristian Leo

Applied Scientist
AWS Security

Daniel Begimher

Senior Security Engineer
AWS Security

AGENDA

01 Threat modeling today

Practice, frameworks, statements, maps

02 Inside ThreatForest

Agents, trust boundaries, gates, evidence

03 Demo

Repository → reviewable attack graph

SECTION 01

THREAT MODELING TODAY

HOW ARE YOU THREAT MODELING TODAY?

Hands up: which description is closest?

A

We threat-model every release

B

We do it once, usually at design time

C

We know we should - but rarely have time

THREAT MODELING TODAY

TRADITIONAL / WORKSHOP-LED

Context-rich, but time- and expertise-intensive

01

Understand the system

SME interviews • architecture • deployment context

02

Apply a framework

STRIDE • PASTA • LINDDUN

03

Prioritize threats & controls

Human judgment • ownership • accountability

COMMON ONE-SHOT LLM

One prompt, one answer, little shared context

01

Ask once

"Threat model this app."

02

Get an instant answer

Threats • mitigations • recommendations

03

Use the draft

No SME interview • limited deployment context

THE OPPORTUNITY: HUMAN CONTEXT AT MACHINE SPEED

A THREAT STATEMENT CAPTURES THE RISK

THREAT STATEMENT



An **unauthorized user** could supply a **URL** that causes the web application to request an **unintended internal or metadata endpoint**, potentially exposing **sensitive data or temporary credentials**.



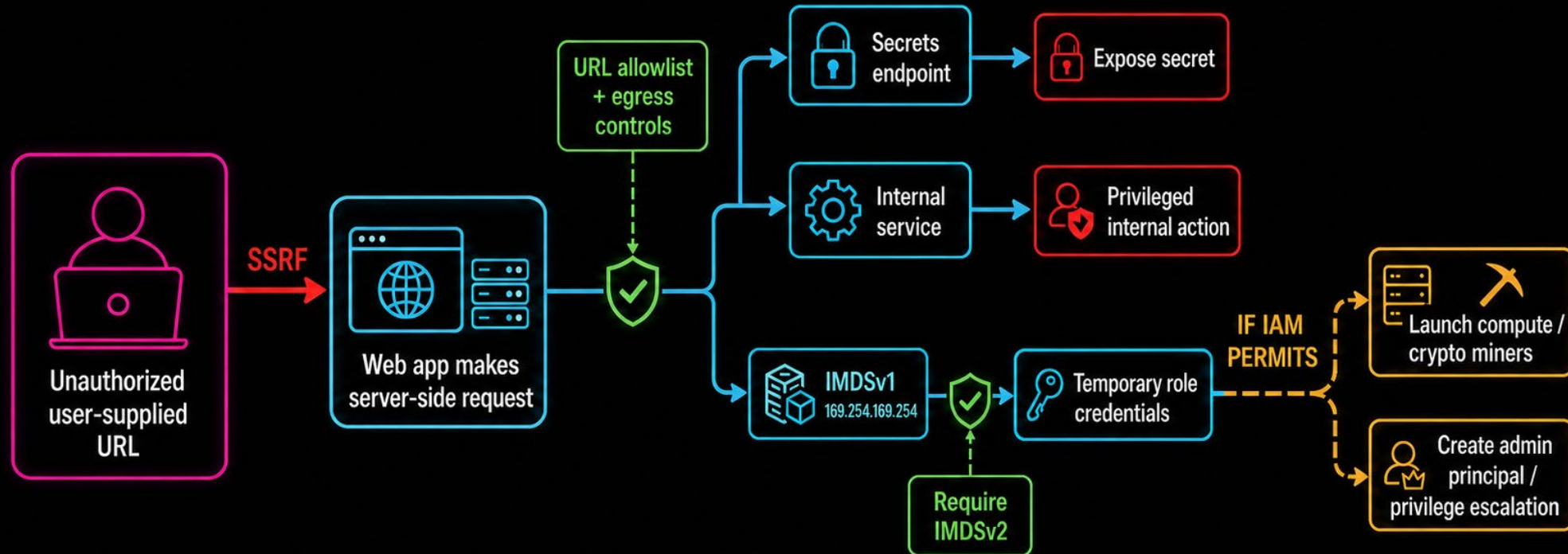
THREAT STATEMENT CAPTURES

actor • action • asset • impact

ATTACK MAP ADDS

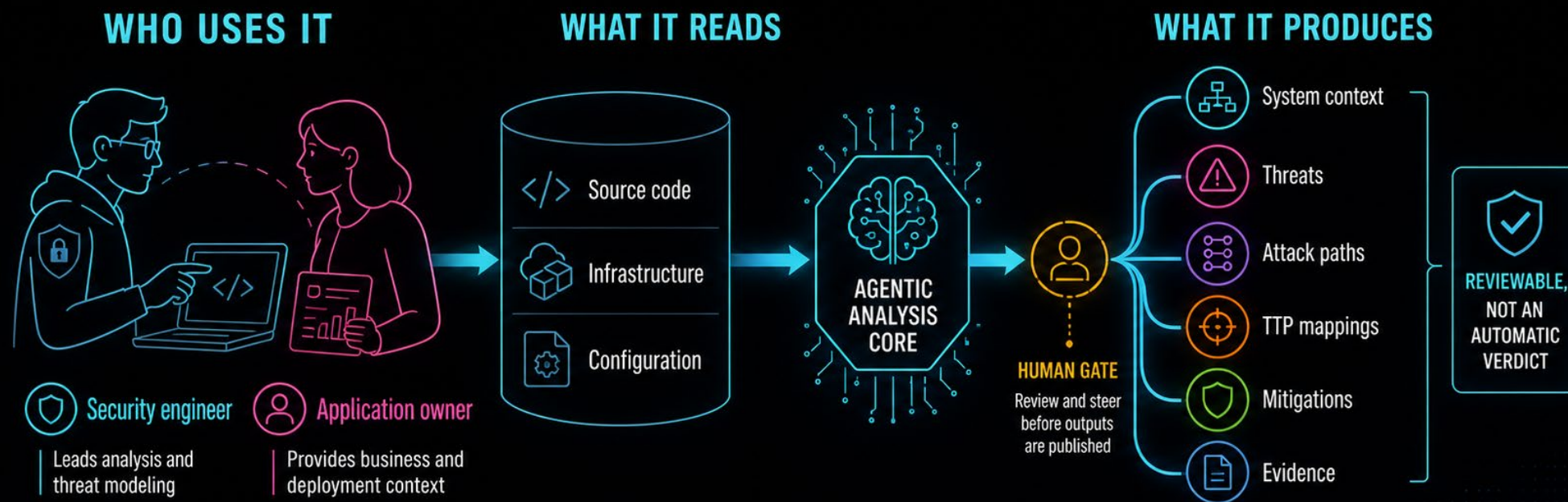
branching paths • dependencies • choke points • downstream actions

AN ATTACK MAP SHOWS HOW SSRF COMPOUNDS



THE STATEMENT NAMES THE RISK. THE MAP REVEALS HOW IT COMPOUNDS.

WHAT THREATFOREST DOES



THREATFOREST TURNS A REPOSITORY INTO A **REVIEWABLE ATTACK MAP**.



SECTION 02

INSIDE THREATFOREST

AUTOMATION NEEDS CONTEXT

AUTOMATION CONTRIBUTES

- 01** Continuous analysis
- 02** Repeatable results
- 03** Repository awareness

HUMANS CONTRIBUTE

- 01** Deployment reality
- 02** Business impact
- 03** Risk appetite

THE THREATFOREST PIPELINE

Agents perform the repeatable analysis. Humans authorize contextual decisions.

01 Understand the system

Agent + deterministic verifier

02 Review context

Human gate

03 Formulate threats

Agent + deterministic verifier

04 Review threats

Human gate

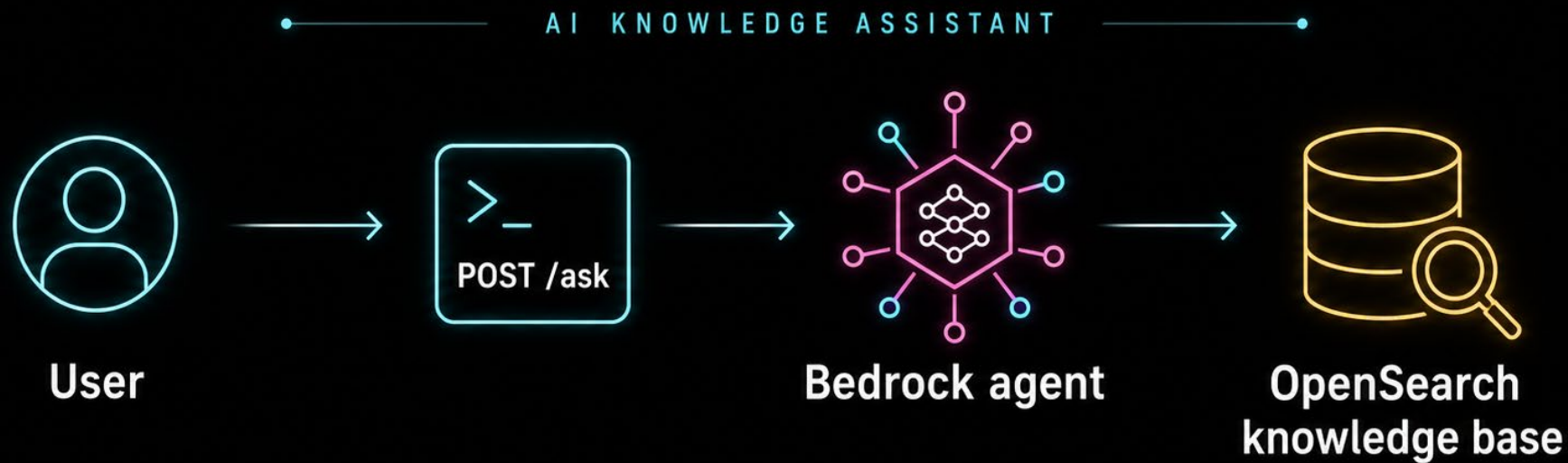
05 Build attack maps

Agent + deterministic verifier

06 Recommend controls

Agent + deterministic verifier

RUNNING EXAMPLE



Answers questions using product manuals

BUILD A DRAFT MODEL

REPOSITORY EVIDENCE

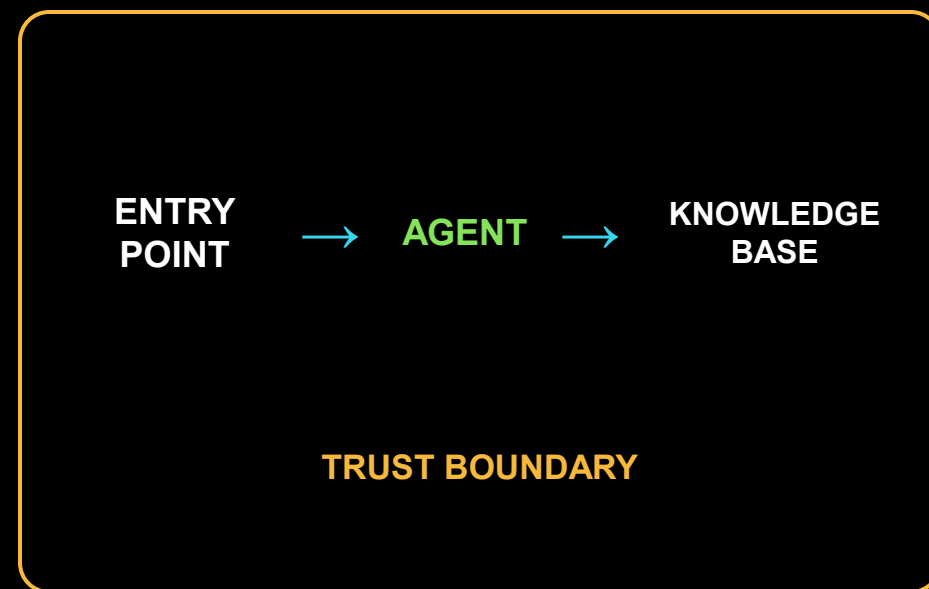
- 01 Source code
- 02 Infrastructure
- 03 Configuration

AGENT



Extracts relationships

DRAFT SYSTEM MODEL



REVIEW CONTEXT

The interviewer asks only for what the repository cannot establish.

01 What lifecycle stage is this in?

Production, early design, or early development?

02 What infrastructure or controls live outside the repository?

Gateways, WAF, service authentication, runtime secrets

03 Who can access it—and how?

Public, internal, multi-tenant; SSO, MFA, roles

04 What blind spots could the scanner not reach?

Operations, incidents, external dependencies

The agent asks 2–3 targeted follow-ups only when critical gaps remain.

FORMULATE THREATS

THREAT STATEMENT GENERATION

01 Identify the actor

Add the access or condition
required to act

02 Trace the action

Connect the abuse to the system
behavior it triggers

03 Name the impact

State the affected asset and
reduced CIA objective

RUNNING EXAMPLE

A **malicious user** with authenticated access to **POST /ask** can **inject instructions** that cause the **Bedrock agent** to retrieve **proprietary manuals**, which leads to **unauthorized disclosure**, resulting in reduced **confidentiality** of those manuals.

REVIEW THREATS

The reviewer decides what should enter deeper attack-path analysis.

01 Do these threats make sense for your application?

02 Do you want to change the priority of any of them?

03 Are there any false positives?

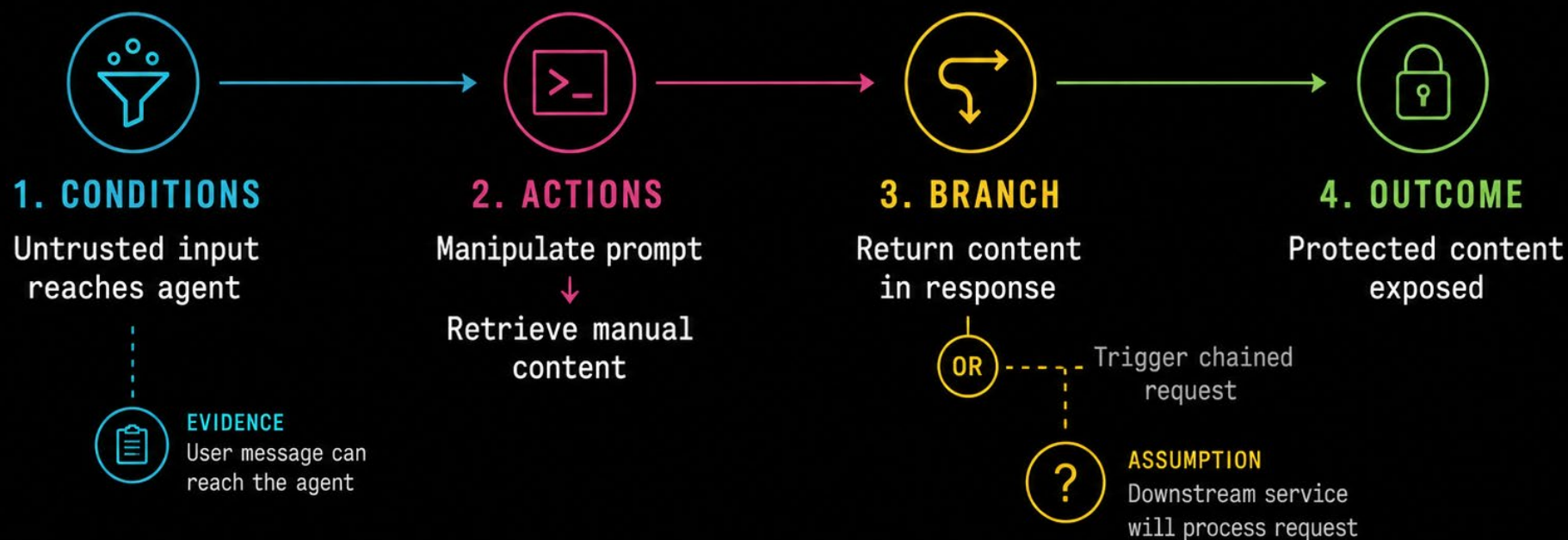
04 Are there any new threats we should add?

Reprioritize • remove • add • proceed

BUILD THE ATTACK GRAPH

APPROVED THREAT STATEMENT

A malicious user with authenticated access to POST /ask can inject instructions that cause the Bedrock agent to retrieve proprietary manuals, which leads to unauthorized disclosure, resulting in reduced confidentiality of those manuals.



WHY TACTICS, TECHNIQUES, AND PROCEDURES MATTER

Mappings turn each attack-tree step into reusable security knowledge.

01 Explain execution

Show how an attacker could carry out the step—not only what outcome they want.

02 Surface novelty signals

A weak or missing catalog match flags behavior that deserves expert review.

03 Expand the attack surface

Related techniques reveal alternative routes, prerequisites, and adjacent behaviors.

04 Connect mitigations

Technique mappings lead to relevant controls, evidence, telemetry, and tests.

CALCULATE FEASIBILITY

Each attack step gets a probability. The complete path compounds them.

01 FACTOR PRIOR

$$p_0 = \sigma(-0.5 + \text{factors})$$

Skill required • access required
Detectability • exploit maturity

02 EVIDENCE UPDATE

$$p = \sigma(\text{logit}(p_0) + \text{evidence})$$

TTP similarity can raise or lower the score.
Mitigations and contradictory evidence lower it.

03 PATH REACH

$$\text{reach}(\text{child}) = p(\text{child}) \times \text{reach}(\text{parent})$$

The fact node starts at 1.0.
Every required step compounds the chain.

EXAMPLE PATH

$$0.88 \times 0.72 \times 0.55 = 0.35$$

35% estimated reach to the outcome

**MODEL ESTIMATE
NOT EXPLOIT PROOF**

GENERATE MITIGATIONS

CANDIDATE CONTROLS

- 01** Isolate untrusted input
- 02** Limit retrieval scope
- 03** Filter sensitive output

RANK IN CONTEXT

- 01** Path impact
- 02** Evidence confidence
- 03** Implementation cost

DEMO: CODE → TREE

KEY TAKEAWAYS

A reviewable foundation developers can build on.

1

See the whole path. Individual risks combine into reachable attack paths.

2

Trust the foundation. Code, evidence, and assumptions remain traceable.

3

Scale expert judgment. Developers can reuse and extend SME reasoning.

Thank you

SCAN FOR THREATFOREST



THREATFOREST

Cristian Leo

Applied Scientist
AWS Security



cristian-leo

Daniel Begimher

Senior Security Engineer
AWS Security



begimher